Datastructures and Algorithms

For week 4 I used binary search. Binary search only works on sorted arrays. It begins by defining three variables: L which is the left most index which always is 0, R which is the right most value which is the length of the array – 1 and M which is the floor of half of the array length. The search follows these 4 steps:

1. If the value on position M is the value it is searching for, return M.
2. If the value on position M is bigger than the value it is searching for, set R to M – 1
3. If the value on position M is smaller than the value it is searching for, set L to M + 1
4. set M to L + (R-L)/2 and repeat step 1.

The suspects that were found using this search were:

- Alfred Pugsley
- Helena Blackwood
- Jesper Wellington
- Nem Wellington

For week 5 I implemented a Binary Search Tree. First, I created a class for both the tree and the nodes in the tree. The tree class contained the builder methods, the insert methods and the search methods.

The node class contained a List of the data it had to contain, a right node, a left node, a builder method and a add method to the list. The reason for the list was to simplify the search. If multiple nodes had the same value in a normal tree, they could go a long way down the tree and create a huge height which is not necessary. The disadvantage of having lists in the nodes is that the insert method and the builder method with initial data will take more time as it has the add the data to a list. The advantage of having a list in the node is that the search is much faster since it does not have to look through the entire tree to find all values within a range. All workers that share the same entrance date are stored in the same node so the search can easily go on for how much it must. In normal cases I believe that the advantage outweighs the disadvantages since reading happens more than writing and reading must go fast.

The Search method for the tree was DFS. The search starts by looking to the left node from the root node to see if it finds the lowest value. This continues for every node it scans: Looks at the left node > goes to the left node > looks at that node's left node. When it finds the lowest value of the range it goes to that right node and repeats the previous steps until it finds a left node which is equal to null. Then it goes to every right node and repeats the previous steps. It does this until it finds the high value of the range and returns everything as an Array of the workers.

A major difficulty during this course was creating the heap sort. The difficulty lay in coding from visualizing the tree. It was very easy to paint how it should work but converted from a visual math tree to code with a 1d array. A problem that occurred was that sometimes the array would not be sorted, even though the code seemed right. It had trouble switching the values when they were supposed to switch. This happened because a less than comparator (<) was a more than comparator (>) which was an easy fix but hard to find. Another problem that occurred was that the lowest value was put in the last index, but the array was sorted except for that. The problem arose because it was missing an iteration where it swaps the highest value with the last index and would instead be swapped with the second last index. The solution was to add an iteration.

The binary search tree search method was the most difficult assignment because finding a range within it proved to be difficult. To simplify the process, as mentioned earlier, the tree stored workers with the same entrance date in the same node. However, a major problem that occurred was a null reference error. It would compare a node that was null and thrown an error. The problem lay in which node it compared. In the code it thought it was going to send the right node when it sent the left node which could be null. This would also return the wrong nodes within the range, but it was a fast fix: switch the node from left to right.